

StepPPO-v2: GiGPO-Aligned Step-wise PPO with a Shared Value Head

Abstract

StepPPO-v2 is a shared-backbone actor-critic method for multi-step agentic reinforcement learning. An episode contains multiple environment interaction steps, and each step produces one complete language-model action. StepPPO-v2 defines credit assignment over environment steps rather than response tokens. The method keeps the GiGPO-style group-relative episode signal, adds a learned step-wise temporal correction from a shared value head, and optimizes the language model with the standard token-level PPO ratio. The value head is attached to the actor backbone; no separate critic model is maintained.

1 Agentic Rollout Formulation

We consider tasks where each sampled episode contains a sequence of agent-environment interaction steps. At environment step t , the agent receives context s_t , generates an action a_t , receives scalar reward r_t , and transitions to s_{t+1} . A trajectory is

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_{T-1}, a_{T-1}, r_{T-1}). \quad (1)$$

The action a_t is a complete response generated by the language model,

$$a_t = (y_{t,1}, y_{t,2}, \dots, y_{t,L_t}), \quad (2)$$

but StepPPO-v2 treats the whole response as a single environment-level action. Thus value estimation, temporal-difference errors, returns, and GAE are computed on the environment-step axis.

Each collected step sample i stores three identifiers:

- u_i : the group id, corresponding to `uid`. It identifies repeated rollouts from the same initial task or prompt.
- τ_i : the trajectory id, corresponding to `traj_uid`. It identifies one sampled episode.
- t_i : the within-trajectory step index, corresponding to `step_idx`. It orders samples inside the same trajectory.

The temporal transition for step-wise GAE is defined by grouping samples with the same τ_i and sorting them by t_i .

2 Shared-Backbone Value Head

Let π_θ be the actor language model. StepPPO-v2 attaches a small value head v_ψ to the actor backbone. Given the tokenized context-action sequence for a step, the actor produces final-layer hidden states

$$H_i = f_\theta(s_i, a_i) \in \mathbb{R}^{n_i \times d}, \quad (3)$$

where n_i is the full sequence length and d is the hidden size.

StepPPO-v2 uses the pre-action boundary hidden state to produce a strict state-value estimate. If the generated action has response length L_i , the value index is

$$b_i = n_i - L_i - 1. \quad (4)$$

The predicted state value is

$$V_{\theta,\psi}(s_i) = v_\psi(H_i[b_i]). \quad (5)$$

Because the backbone is causal, $H_i[b_i]$ cannot attend to generated action tokens. The value is therefore a baseline for the state before action generation.

The value head is an MLP regression head:

$$v_\psi(h) = W_2\phi(W_1h + b_1) + b_2, \quad (6)$$

where ϕ is GELU. StepPPO-v2 does not detach the value head from the actor backbone:

$$\nabla_\theta \mathcal{L}_v \neq 0. \quad (7)$$

Thus the value loss updates both the value-head parameters ψ and the shared actor backbone θ . To reduce interference from value regression, StepPPO-v2 uses a smaller value-loss coefficient.

3 GiGPO-Aligned Episode Advantage

For each step sample i , let S_i denote the episode-level score. In implementation, S_i is read from the collected episode return field, so every valid step in the same sampled episode receives the same episode score. Equivalently,

$$S_i = \sum_{t=0}^{T(\tau_i)-1} r_{\tau_i, t}. \quad (8)$$

StepPPO-v2 aligns the episode advantage with the WebShop GiGPO baseline, which uses `mean_norm`. For each group id u , define the row-level group set

$$\mathcal{I}_u = \{i \in \{1, \dots, N\} : u_i = u\}, \quad n_u = |\mathcal{I}_u|. \quad (9)$$

The group mean is

$$\mu_u = \frac{1}{n_u} \sum_{j \in \mathcal{I}_u} S_j. \quad (10)$$

The episode advantage is the group-centered score

$$A_i^{\text{epi}} = S_i - \mu_{u_i}. \quad (11)$$

No additional batch-level whitening is applied to A_i^{epi} . This keeps the episode term on the same normalization semantics as GiGPO.

4 Step-wise Critic Return and Advantage

Before PPO updates, StepPPO-v2 evaluates the behavior policy once to obtain both old token log probabilities and old state values:

$$\log \pi_{\theta_{\text{old}}}(a_i | s_i), \quad V_i^{\text{old}} = V_{\theta_{\text{old}}, \psi_{\text{old}}}(s_i). \quad (12)$$

For a step sample $i = (\tau, t)$, let $i^+ = (\tau, t + 1)$ be the next step from the same trajectory if it exists. Let $d_i = 1$ if i is the final valid step in the trajectory and $d_i = 0$ otherwise. The step-wise TD residual is

$$\delta_i = r_i + \gamma(1 - d_i)V_{i^+}^{\text{old}} - V_i^{\text{old}}. \quad (13)$$

The raw step advantage is computed by GAE on the environment-step axis:

$$A_i^{\text{step, raw}} = \delta_i + \gamma\lambda(1 - d_i)A_{i^+}^{\text{step, raw}}. \quad (14)$$

The value target is

$$R_i^{\text{step}} = A_i^{\text{step, raw}} + V_i^{\text{old}}. \quad (15)$$

StepPPO-v2 standardizes the step advantage over all valid step samples in the current training batch:

$$\hat{A}_i^{\text{step}} = \frac{A_i^{\text{step, raw}} - \mu_{\mathcal{B}}^{\text{step}}}{\sigma_{\mathcal{B}}^{\text{step}} + \epsilon}, \quad (16)$$

where

$$\mu_{\mathcal{B}}^{\text{step}} = \frac{1}{N} \sum_{i=1}^N A_i^{\text{step, raw}}, \quad \sigma_{\mathcal{B}}^{\text{step}} = \sqrt{\frac{1}{N} \sum_{i=1}^N (A_i^{\text{step, raw}} - \mu_{\mathcal{B}}^{\text{step}})^2}. \quad (17)$$

This standardization controls the scale of the learned critic correction before it is combined with the GiGPO-aligned episode term.

5 Final Advantage

The final StepPPO-v2 policy advantage is

$$A_i^{v2} = A_i^{\text{epi}} + w\hat{A}_i^{\text{step}}, \quad (18)$$

with fixed coefficient

$$w = 0.5. \quad (19)$$

The combined advantage is not divided by $1 + w$ and is not whitened after combination. It is broadcast to valid response tokens:

$$A_{i,\ell}^{v2} = A_i^{v2} m_{i,\ell}, \quad (20)$$

where $m_{i,\ell}$ is the response or loss mask. The step return is also broadcast for compatibility with the existing token-shaped value-loss path:

$$R_{i,\ell}^{\text{step}} = R_i^{\text{step}} m_{i,\ell}. \quad (21)$$

6 Optimization Objective

For each response token, define the PPO ratio

$$\rho_{i,\ell}(\theta) = \exp(\log \pi_\theta(y_{i,\ell} \mid s_i, y_{i,<\ell}) - \log \pi_{\theta_{\text{old}}}(y_{i,\ell} \mid s_i, y_{i,<\ell})). \quad (22)$$

The clipped policy loss is

$$\mathcal{L}_{\text{pg}}(\theta) = -\frac{1}{\sum_{i,\ell} m_{i,\ell}} \sum_{i=1}^N \sum_{\ell=1}^L m_{i,\ell} \min(\rho_{i,\ell}(\theta) A_i^{v2}, \text{clip}(\rho_{i,\ell}(\theta), 1 - \epsilon_p, 1 + \epsilon_p) A_i^{v2}). \quad (23)$$

For value learning, let

$$V_i = V_{\theta,\psi}(s_i), \quad \tilde{V}_i = V_i^{\text{old}} + \text{clip}(V_i - V_i^{\text{old}}, -\epsilon_v, \epsilon_v). \quad (24)$$

The clipped value loss is

$$\mathcal{L}_v(\theta, \psi) = \frac{1}{2N} \sum_{i=1}^N \max((V_i - R_i^{\text{step}})^2, (\tilde{V}_i - R_i^{\text{step}})^2). \quad (25)$$

The total update loss is

$$\mathcal{L}(\theta, \psi) = \mathcal{L}_{\text{pg}}(\theta) + c_v \mathcal{L}_v(\theta, \psi) + c_{\text{kl}} \mathcal{L}_{\text{kl}}(\theta) - c_H \mathcal{H}(\theta). \quad (26)$$

StepPPO-v2 uses

$$c_v = 0.03, \quad \epsilon_v = 0.5. \quad (27)$$

7 Algorithm

8 Implementation Configuration

The StepPPO-v2 method is implemented with the shared value-head actor path. The deterministic paper configuration is:

```
algorithm.step_ppo.step_advantage_w=0.5
algorithm.step_ppo.episode_mode=mean_norm
algorithm.step_ppo.normalize_episode_advantage=False
algorithm.step_ppo.normalize_step_advantage=True
algorithm.step_ppo.normalize_final_advantage=False
actor_rollout_ref.actor.value_head.detach_value_backbone=False
actor_rollout_ref.actor.value_head.value_loss_coef=0.03
actor_rollout_ref.actor.value_head.cliprange_value=0.5
actor_rollout_ref.actor.ppo_mini_batch_size=64
actor_rollout_ref.actor.ppo_micro_batch_size_per_gpu=8
actor_rollout_ref.rollout.log_prob_micro_batch_size_per_gpu=16
actor_rollout_ref.ref.log_prob_micro_batch_size_per_gpu=16
```

Algorithm 1 StepPPO-v2 training iteration

Input: Actor π_θ with value head v_ψ ; grouped prompts; rollout repeat count K ; $\gamma, \lambda, w = 0.5, c_v = 0.03$.**Output:** Updated actor and value head.

- 1: Sample initial tasks and repeat each task K times.
 - 2: Roll out multi-step trajectories with the behavior policy $\pi_{\theta_{\text{old}}}$.
 - 3: Store each valid action step as $(u_i, \tau_i, t_i, s_i, a_i, r_i, S_i, m_i)$.
 - 4: Run one old-policy forward pass to compute old token log probabilities and old state values V_i^{old} .
 - 5: Compute $A_i^{\text{epi}} = S_i - \mu_{u_i}$ within each `uid` group.
 - 6: Group by `traj_uid`, sort by `step_idx`, and compute $A_i^{\text{step,raw}}$ and R_i^{step} .
 - 7: Standardize $A_i^{\text{step,raw}}$ over the batch to obtain $\hat{A}_i^{\text{step}}$.
 - 8: Form $A_i^{v2} = A_i^{\text{epi}} + 0.5\hat{A}_i^{\text{step}}$.
 - 9: Broadcast A_i^{v2} and R_i^{step} to response-token shape with the loss mask.
 - 10: **for** each PPO epoch and minibatch **do**
 - 11: Recompute current token log probabilities and current values V_i .
 - 12: Minimize $\mathcal{L}_{\text{pg}} + 0.03\mathcal{L}_v + c_{\text{kl}}\mathcal{L}_{\text{kl}} - c_H\mathcal{H}$.
-

The actor update micro-batch size is set to 8 per GPU to match the GiGPO WebShop baseline. The rollout and reference log-probability micro-batches remain at 16 per GPU, also matching the GiGPO script.

9 Implementation Mapping

Component	Code path	StepPPO-v2 behavior
Value head	<code>value_head.py</code>	Attach MLP value head to actor hidden states
Actor path	<code>step_ppo_actor.py</code>	Compute token log probabilities and state values in actor forward
Old values	<code>compute_log_prob</code> path	Return old log probabilities and old state values together
Step GAE	<code>step_ppo_algos.py</code>	Group by <code>traj_uid</code> , sort by <code>step_idx</code>
Episode term	advantage function	Use <code>uid-group</code> GiGPO <code>mean_norm</code>
Final advantage	advantage function	Use $A^{\text{epi}} + 0.5\hat{A}^{\text{step}}$
Value loss	actor update	Use clipped value loss with $c_v = 0.03$ and no detach
Micro-batch	training script	Set actor PPO micro-batch to 8 per GPU

10 Differences from StepPPO-v1

StepPPO-v2 differs from StepPPO-v1 only in the following deterministic design choices:

- **Episode advantage:** v1 uses GRPO-style episode advantage followed by extra batch-level whitening. v2 uses GiGPO-aligned `uid-group mean_norm` and does not apply additional batch-level whitening to the episode term.
- **Final advantage:** v1 uses $(\hat{A}^{\text{epi}} + w\hat{A}^{\text{step}})/(1 + w)$. v2 uses direct addition, $A^{\text{epi}} + w\hat{A}^{\text{step}}$.
- **Step weight:** v1 uses $w = 1.0$. v2 uses $w = 0.5$.
- **Value loss coefficient:** v1 uses $c_v = 0.1$. v2 uses $c_v = 0.03$.
- **Value backbone gradient:** v1 already allows value-loss gradients to update the shared backbone. v2 keeps this choice with `detach_value_backbone=False`.
- **Micro-batch:** v1 WebShop scripts used actor PPO micro-batch size 4 per GPU. v2 aligns with GiGPO and uses actor PPO micro-batch size 8 per GPU.

- **Old-value collection:** v2 keeps the single-forward path that returns old token log probabilities and old state values together when the value head is enabled.